

## FORMATION PYTHON

# J4

## Objets et analyse de données

Dataclasses complètes, sélection loc/iloc, dates, groupby  
et merge en détail — puis lancement du projet final.

08-poo

09-pandas-analyse

projet-final

# Objectifs de la journée



## Modéliser avec des dataclasses

field, frozen, \_\_post\_init\_\_, property : données + règles ensemble.



## Nettoyer avec pandas

dtypes, doublons, manquants, libellés, dates — sur 60 000 lignes.



## Agréger et joindre

groupby/agg, merge (how, indicator) : déclarations × contribuables × régions.



## Lancer le projet

Tirage des sujets en fin de matinée — 5 sujets, groupes de 2-3.

### LA SEMAINE

J1

Fondamentaux du langage

J2

Fonctions & structures

J3

Fichiers & robustesse

J4

**Objets & pandas**

J5

Intégration & projets

*Après-midi : premier jalon du projet.*

# @dataclass : la fiche qui se défend



## **\_\_init\_\_ et \_\_eq\_\_ écrits pour vous**

Les champs annotés suffisent — et deux déclarations égales se comparent par valeur.



## **@property : valeur calculée**

d.tva\_nette se LIT comme un champ, se CALCULE comme une méthode — jamais désynchronisée.



## **field(default\_factory=list)**

Le défaut mutable du J2, résolu proprement pour les listes de déclarations.



## **Annotations obligatoires**

periode: str n'est pas décoratif : c'est ce que @dataclass lit pour générer le code.



tp\_contribuable.py

```
@dataclass
class Declaration:
    periode: str
    chiffre_affaires: int
    tva_collectee: int
    tva_deductible: int

    @property
    def tva_nette(self) -> int:
        return max(self.tva_collectee
                    - self.tva_deductible, 0)
```

# dataclass avancé : field, frozen, \_\_post\_init\_\_



## **frozen=True fige l'objet**

Un référentiel (code, nom de région) ne doit pas bouger — l'écriture lève une erreur.



## **\_\_post\_init\_\_ valide à la création**

Un CA négatif est refusé À LA SOURCE — plus aucun objet invalide en circulation.



## **field(repr=False) allège**

La liste des 36 déclarations n'encombre plus l'affichage de la fiche.



## **Comparer, trier**

order=True génère <, <= : des objets triables par leurs champs.



validation à la source

```
@dataclass(frozen=True)
class Region:
    code: str
    nom: str

@dataclass
class Declaration:
    periode: str
    chiffre_affaires: int

    def __post_init__(self):
        if self.chiffre_affaires < 0:
            raise ValueError("CA < 0")
```

# Méthodes : le comportement près des données



## **self = l'objet courant**

c.ajouter(d) appelle ajouter(c, d) — self n'a rien de magique.



## **Les méthodes composent les propriétés**

tva\_due\_totale() somme les tva\_nette : la règle « jamais négative » s'applique partout.



## **\_\_repr\_\_ gratuit et lisible**

print(c) affiche Contribuable(nif='...', ...) — le débogage est déjà servi.



## **Quand NE PAS faire de classe**

Aucune règle à protéger, données de passage ? Un dict suffit — la classe se mérite.



comportements

```
@dataclass
class Contribuable:
    nif: str
    declarations: list[Declaration] = \
        field(default_factory=list)

    def ajouter(self, d): ...
    def tva_due_totale(self) -> int:
        return sum(d.tva_nette
                    for d in self.declarations)

c.declarations_manquantes(2025)
['2025-03', '2025-07', ...]
```

# pandas : charger, typer, inspecter



## Le DataFrame : un tableau typé

Colonnes nommées, opérations vectorisées — fini la boucle ligne à ligne.



## dtype={"nif": str} au chargement

Sinon les NIF deviennent des int64 et perdent leurs zéros de tête — irréversible.



## info() : types ET manquants

La radiographie : 60 000 lignes, tva\_collectee non-null 58 198 → 1 802 trous.



## describe() : ordres de grandeur

min négatif sur chiffre\_affaires ? Les aberrations sautent aux yeux AVANT le calcul.



tp\_analyse.py

```
import pandas as pd

decl = pd.read_csv(
    DATA / "declarations.csv",
    dtype={"nif": str})

decl.info()
chiffre_affaires    60000 non-null
tva_collectee       58198 non-null

decl["chiffre_affaires"].describe()
min    -4.98e+08    # aberrant !
```

# Sélectionner : masques, loc, iloc



## Le masque booléen filtre

`df[df["chiffre_affaires"] > 0]` — la condition indexe le tableau.



## Conditions combinées : & | et ( )

`(df["ca"] > 0) & (df["periode"] >= "2025-01")` — parenthèses OBLIGATOIRES.



## loc = étiquettes, iloc = positions

`df.loc[m, "tva_nette"]` cible lignes ET colonnes ; `df.iloc[:5, :3]` coupe par position.



## Écrire via loc, jamais en chaîne

`df.loc[m, "col"] = v` — `df[m]["col"] = v` modifie une COPIE (SettingWithCopyWarning).



masques & loc

```
valides = decl[decl["chiffre_affaires"] > 0]
```

```
m = (decl["chiffre_affaires"] > 0) & \
    (decl["periode"] >= "2025-01")
```

```
recent = decl.loc[m,
    ["nif", "tva_collectee"]]
```

```
# écrire : loc, jamais en chaîne
decl.loc[decl["tva_collectee"].isna(),
    "tva_collectee"] = 0
```

# Nettoyer : tout le J3, vectorisé



## **drop\_duplicates(subset="nif")**

Les doubles saisies disparaissent — keep="first" garde la première.



## **.str : les méthodes de chaînes en colonne**

.str.strip().str.upper() : « commerce gal » et « COMMERCE GENERAL » fusionnent.



## **fillna(0) après réflexion**

Une TVA absente vaut-elle 0 ? Ici oui (non déclarée) — ce choix se documente.



## **clip(lower=0)**

La TVA nette ne descend jamais sous zéro — la règle du module 08, en colonne.



nettoyage

```
contr = contr.drop_duplicates(
    subset="nif")

contr["secteur"] = (contr["secteur"]
    .str.strip().str.upper())

decl["tva_collectee"] = \
    decl["tva_collectee"].fillna(0)

decl["tva_nette"] = (decl["tva_collectee"]
    - decl["tva_deductible"]).clip(lower=0)
```



# Les dates : to\_datetime et l'accessor .dt



## to\_datetime convertit la colonne

format="mixed" absorbe nos quatre formats ; errors="coerce" → NaT au lieu de planter.



## NaT : le manquant des dates

isna() les compte — ce sont nos « dates inexploitable » du J3, version pandas.



## .dt expose les composants

.dt.year, .dt.month, .dt.to\_period("M") : la période mensuelle en une ligne.



## Comparer des dates devient trivial

df[df["date\_depot"] > "2025-06-30"] : les dépôts du second semestre.



dates

```
decl["date_depot"] = pd.to_datetime(
    decl["date_depot"],
    format="mixed", dayfirst=True,
    errors="coerce") # NaT si échec

decl["date_depot"].isna().sum()
142 # inexploitable

decl["mois"] = decl["date_depot"]
    .dt.to_period("M")
2025-03, 2025-04, ...
```

# groupby : éclater, agréger, recomposer



## Le comptage du J2, généralisé

groupby("region")["tva\_nette"].sum() remplace le dict + get(k,0).



## agg pour plusieurs mesures

agg(total=..., moyenne=..., n=...) : trois agrégats nommés en un passage.



## Grouper par plusieurs clés

groupby(["region", "secteur"]) : le croisement, trié par sort\_values.



## reset\_index() pour ré-aplatir

Le résultat redevient un DataFrame ordinaire, prêt à joindre ou exporter.



agrégats nommés

```
rapport = (ens
    .groupby(["region", "secteur"])
    .agg(
        total=("tva_nette", "sum"),
        moyenne=("tva_nette", "mean"),
        n=("id_declaration", "count"))
    .reset_index()
    .sort_values("total",
        ascending=False))
```

```
Gabú    MINES    6.84e+09    2.1e+06    3204
```

# merge : la jointure, et ses révélations



## **on= la clé, how= la stratégie**

inner garde les communs ; left garde TOUTES les déclarations, même orphelines.



## **indicator=True trace l'origine**

\_merge vaut left\_only pour une déclaration sans contribuable — le comptage est offert.



## **validate= vérifie la cardinalité**

"m:1" : plusieurs déclarations, UN contribuable — sinon erreur immédiate (doublons !).



## **Dédoublonner AVANT de joindre**

Un référentiel avec doublons multiplie les lignes en silence — le piège classique du merge.



jointures

```
ens = decl.merge(contr, on="nif",  
                 how="left", indicator=True,  
                 validate="m:1")  
  
ens["_merge"].value_counts()  
both          58 706  
left_only      1 294 # orphelines  
  
ens = ens.merge(  
    regions[["code_region", "region"]]  
    .drop_duplicates(),  
    on="code_region", how="left")
```

# Cinq sujets, tirés au sort

## 1 • Défaillants

Qui n'a rien déposé sur la période ? Par région, par secteur.

## 2 • Crédits de TVA

Où la déductible dépasse-t-elle durablement la collectée ?

## 3 • Recouvrement

Déclarations × paiements : taux de couverture, restes à payer.

## 4 • Qualité des données

Mesurer et corriger : doublons, dates, libellés, orphelins.

## 5 • Tableau de bord régional

CA, TVA, top secteurs, évolution — export Excel multi-feuilles.

# Les jalons — et le premier est aujourd'hui



## J4 midi

Sujet tiré, dépôt du groupe créé,  
socle exécuté tel quel



## J4 soir

Lecture + validation  
opérationnelles sur les vraies  
données



## J5 midi

Chaîne complète exécutable en  
UNE commande



## J5 après-midi

Soutenance : 10 min de  
démonstration + 5 min de  
questions

**Règle absolue :** les lignes écartées sont comptées et affichées — jamais perdues en silence. Le programme échoue proprement si les fichiers manquent.

# Objets, pandas — et le projet démarre

1 **tp\_contribuable.py** — tva\_nette, ca\_total, declarations\_manquantes — assert fournis

2 **tp\_analyse.py** — nettoyage, jointures validées, rapport TVA exporté

3 **Tirage des sujets** — fin de matinée — constituez les groupes AVANT

4 **Jalon du soir** — socle exécuté + lecture/validation sur vraies données



## Fichiers du jour

```
08-poo/  
    tp_contribuable.py  
09-pandas-analyse/  
    tp_analyse.py  
projet-final/consignes.md
```



Comparez vos comptages pandas avec ceux du J3 « à la main » : mêmes chiffres = confiance.

# À retenir

- ✓ **dataclass complète** — `field`, `frozen`, `__post_init__` : plus aucun objet invalide en circulation.
- ✓ **`dtype={"nif": str}` et `loc`** — les deux pièges pandas majeurs — identifiants et écriture — sont désamorçés.
- ✓ **`to_datetime + .dt`** — les quatre formats du J3 absorbés en une ligne, NaT comptés.
- ✓ **`agg` nommé + `merge` validé** — `indicator` révèle les orphelines, `validate="m:1"` bloque les doublons.
- ✓ **Le projet est lancé** — sujet tiré, socle en main, premier jalon franchi ce soir.



## Demain — J5 : intégration et soutenances

`argparse`, `logging`, `vectorisation`, `checklist` qualité — la chaîne complète en une commande, puis les soutenances.